Mémoire M2 : Autoscaling proactif en microservices — résumé complet

Résumé de travail au 23 octobre 2026, destiné à être transmis à une autre IA pour reprendre exactement au même point.

Contexte et instructions pour l'IA qui reprend

Je suis étudiant en Master 2 Informatique (parcours recherche, Informatique Fondamentale, venant du génie logiciel) à l'Université de Yaoundé I. Dans ce parcours, je dois travailler sur un thème de recherche et le soutenir à la fin. Reprends exactement là où nous nous sommes arrêtés, sans refaire ce qui est déjà acquis.

Consignes de réponse :

- Réponds toujours **en français**.
- Utilise une **notation mathématique formelle et rigoureuse** (∀, Σ, ∈, ⩽, variables indicées, etc.), comme attendu par les évaluateurs.
- Sois structuré, pédagogique, avec des exemples concrets.
- **Vérifie toute référence bibliographique** avant de la présenter comme certaine : les références citées ci-dessous viennent de la mémoire d'une IA et doivent être contrôlées (titre, auteurs, année) sur Google Scholar, ACM DL ou IEEE Xplore.

1. Décision prise : le problème choisi

À partir d'une liste de 10 problèmes de recherche en microservices (cohérence des données, découpage d'un monolithe, anti-patterns, analyse de cause racine, évolution des API, tests, résilience, autoscaling, vérification formelle, sécurité), j'ai **choisi le problème n°8 : autoscaling intelligent et allocation de ressources**.

Énoncé : décider automatiquement, au bon moment, combien d'instances allouer à chaque microservice, en tenant compte des dépendances entre services, pour respecter la performance demandée (SLO) au moindre coût.

2. Explication du problème

Exemple concret (boutique en ligne)

Chaîne : Client → Commande → Paiement → banque externe.

- Une instance de *Commande* traite 100 req/s ; une instance de *Paiement* traite 20 req/s (appel à une banque lente). Chaque commande déclenche un paiement.
- Black Friday : la charge passe de 100 à 400 req/s.
- **Autoscaling naïf (réactif, service par service) :** *Commande* passe à 4 instances, mais *Paiement* reçoit 400 req/s sans avoir réagi. Ses files explosent, timeouts, erreurs ; il ne scale qu'après détection, et le démarrage prend 1 à 2 minutes.
- **Autoscaling intelligent :** le système sait que *Commande* appelle *Paiement* et calcule 400/20 = **20 instances** de *Paiement* nécessaires. Il agit sur les deux services **simultanément**, voire **avant** le pic s'il est prévisible (par exemple tous les vendredis à 20h).

Mots clés définis

- **Microservice / instance / réplica :** un service est une application indépendante ; une instance est une copie en cours d'exécution (souvent un conteneur).
- **Scalabilité :** capacité à supporter plus de charge en ajoutant des ressources.
- **Scaling horizontal (scale out/in) vs vertical (scale up/down) :** ajouter ou retirer des instances, versus donner plus de CPU ou de mémoire à une instance.
- **Autoscaling :** ajustement automatique des ressources. **HPA** (Horizontal Pod Autoscaler de Kubernetes) fonctionne par seuils.
- **Charge de travail (workload) :** trafic demandé au système, variable dans le temps.
- **SLI / SLO / SLA :** indicateur mesuré, objectif interne (par exemple 95 % des requêtes en moins de 300 ms), engagement contractuel.
- **Latence, débit (throughput), saturation.**
- **Sur-provisionnement / sous-provisionnement :** compromis coût/performance.
- **Dépendances entre services, graphe de dépendances (graphe d'appels).**
- **Goulot d'étranglement (bottleneck)** et **propagation en cascade.**
- **Scaling réactif vs proactif (prédictif).**
- **Temps de démarrage (cold start, warm-up).**
- **Oscillation (thrashing) :** ajout puis retrait d'instances en boucle.
- **Allocation de ressources :** répartir un budget limité (CPU, mémoire, machines) entre des services concurrents.
- **Techniques de recherche courantes :** théorie du contrôle, apprentissage par renforcement, séries temporelles (ARIMA, LSTM), théorie des files (M/M/c), optimisation sous contraintes.

3. Solutions existantes et leurs limites

Famille 1 : règles et seuils (industrie)

HPA Kubernetes, KEDA, AWS Auto Scaling, VPA.

Limites : service traité isolément, réactif, seuils manuels et statiques, CPU mauvais proxy de la latence, oscillations.

Famille 2 : prédictif (séries temporelles)

ARIMA, Holt-Winters, LSTM, Google Autopilot (EuroSys 2020).

Limites : fragile face aux pics imprévus et à la dérive de charge (concept drift), prédit la charge d'un service sans modéliser sa propagation, besoin de beaucoup d'historique, erreurs de prévision directement répercutées.

Famille 3 : théorie des files et modèles analytiques

M/M/c, réseaux de Jackson.

Limites : hypothèses irréalistes (arrivées de Poisson, temps de service exponentiels), calibration difficile, passage à l'échelle délicat (appels conditionnels, parallèles, asynchrones).

Famille 4 : conscientes des dépendances et des SLO

Microscaler (IWQoS 2019), Erms (ASPLOS 2023), Seer (ASPLOS 2019), Autothrottle (NSDI 2024). C'est la famille la plus proche de mon sujet.

Limites : traces distribuées coûteuses, topologie supposée plutôt statique, profilage préalable long à refaire à chaque changement de code, peu de prise en compte de l'asynchrone, du cache et des chemins conditionnels.

Famille 5 : apprentissage par renforcement et apprentissage profond

FIRM (OSDI 2020), Sinan (ASPLOS 2021), GRAF (CoNEXT 2021, réseaux de neurones sur graphes).

Limites : coût d'entraînement élevé, faible généralisation entre applications et topologies, exploration risquée en production, manque d'interprétabilité et de garanties, surcharge du contrôleur.

Famille 6 : optimisation et contrôle

MPC, PID/adaptatif, programmation linéaire en nombres entiers.

Limites : modèle simplifié requis, optimisation entière NP-difficile, sensibilité aux retards de démarrage non modélisés.

Synthèse des limites transversales

#	Limite	Familles
L1	Dépendances ignorées ou statiques	1, 2, 3
L2	Réactivité trop tardive (démarrage des instances)	1
L3	Faible robustesse aux pics imprévus et à la dérive	2, 5
L4	Hypothèses de modélisation irréalistes	3, 6
L5	Coût d'apprentissage, de profilage et de monitoring	4, 5
L6	Mauvaise généralisation d'une application à une autre	4, 5
L7	Manque de garanties formelles et d'interprétabilité	5
L8	Contrôleur centralisé : passage à l'échelle, point de défaillance	4, 5, 6

4. Cinq perspectives de solution

Formalisation de base

Soit G = (S, E) le graphe orienté des services, n_i(t) ∈ ℕ le nombre d'instances du service i ∈ S à l'instant t, c_i le coût unitaire d'une instance, λ_i(t) la charge reçue par i, L(t) la latence de bout en bout, α_{ij} le facteur d'appel de i vers j. Objectif :

min Σ*t* Σ{i∈S} c_i · n_i(t), sous : ∀t, L(t) ⩽ SLO, et λ_j(t) = Σ*{(i,j)∈E}* *α*{ij} · λ_i(t).

Perspective	Idée	Limites visées	Nature	Difficulté
P1 Propagation de charge dans le graphe	Estimer les α_{ij} depuis les traces, calculer la charge future induite et scaler tout le chemin simultanément	L1, L2	Algorithmique	Moyenne

Perspective	Idée	Limites visées	Nature	Difficulté
P2 Prédiction hybride robuste	Prévision de fond, détecteur de changement de régime, repli réactif, intervalle de confiance	L2, L3	Algorithmique + statistique	Moyenne
P3 Garanties formelles	Modéliser (réseau de files, automate hybride, système de transitions) et vérifier SLO, absence d'oscillation, borne de coût (model checking statistique)	L4, L7	Théorique / formelle	Élevée
P4 RL + GNN avec transfert	Agent RL sur représentation par réseau de neurones sur graphes, entraîné en simulation, transfert / méta-apprentissage, garde-fous (shielding)	L3, L5, L6	Apprentissage	Élevée
P5 Décentralisé multi-agents	Un agent par service, échange de signaux légers avec les voisins, sans contrôleur central, analyse de convergence	L1, L5, L8	Systèmes distribués	Élevée

Ces perspectives sont combinables (P1 comme brique de base pour P2, P3 ou P5).

5. Ma solution en une phrase (proposition de l'IA, non encore validée)

> Un mécanisme d'autoscaling **proactif** qui, à partir du graphe de dépendances, **propage la charge prévue** d'un service vers tous ceux situés en aval et **dimensionne l'ensemble des services concernés simultanément**, avant que le pic n'arrive, afin de respecter les SLO au moindre coût.

Elle combine P1 et P2. Trois mots d'ordre : **global** (graphe entier), **anticipé** (avant la dégradation), **optimisé** (minimiser le coût sous contrainte de SLO).

6. Thème de recherche proposé (dernière étape réalisée)

Titre (FR) : *Autoscaling proactif et conscient des dépendances pour les architectures microservices : propagation de charge sur le graphe d'appels et allocation de ressources sous contrainte de SLO*

Titre (EN) : *Proactive Dependency-Aware Autoscaling for Microservices Architectures: Load Propagation over the Call Graph and SLO-Constrained Resource Allocation*

Question de recherche

Comment dimensionner dynamiquement et simultanément l'ensemble des services d'une application microservices, en exploitant le graphe de dépendances et la prédiction de charge, de façon à respecter les SLO au moindre coût, malgré l'évolution de la topologie et l'imprévisibilité des pics ?

Formalisation

Pour tout i ∈ S et tout t, trouver n_i(t) ∈ ℕ qui minimise Σ*t* Σ{i∈S} c_i · n_i(t) sous :

- ∀t, L(t) ⩽ SLO ;
- ∀j ∈ S, λ_j(t) = λ_j^ext(t) + Σ*{(i,j)∈E}* *α*{ij} · λ_i(t) ;
- ∀i ∈ S, λ_i(t) ⩽ n_i(t) · μ_i (μ_i : débit d'une instance).

Objectifs

1. Modéliser la propagation de charge et estimer les α_{ij} à partir des traces distribuées.
2. Prévoir la charge d'entrée avec une méthode robuste aux pics et à la dérive, puis la propager vers l'aval.
3. Concevoir un algorithme d'allocation conjointe des n_i(t) qui tient compte du temps de démarrage des instances.
4. Évaluer face aux solutions existantes : violations de SLO, coût, stabilité (absence d'oscillations), coût de calcul du contrôleur.

Hypothèse de travail

Un autoscaler qui propage la charge prévue sur le graphe et agit sur tous les services concernés avant le pic réduit les violations de SLO et le coût par rapport à un autoscaling indépendant par service.

Contributions attendues

- Un modèle de propagation de charge mis à jour en ligne.
- Un algorithme d'allocation proactive globale avec analyse de complexité.

- Une évaluation expérimentale comparative (HPA et au moins une approche de l'état de l'art).

Méthodologie

1. État de l'art (revue systématique).
2. Modélisation (graphe, charge, coût).
3. Conception de l'algorithme (propagation, prévision, allocation).
4. Implémentation d'un prototype sur Kubernetes (contrôleur personnalisé).
5. Expérimentation : benchmarks TrainTicket, Sock Shop ou DeathStarBench ; générateurs de charge Locust ou k6 ; traces réelles publiques (Alibaba, Azure, Google cluster data).
6. Analyse des résultats et discussion des limites.

Mots clés : microservices, autoscaling, allocation de ressources, graphe de dépendances, propagation de charge, SLO, prédiction de charge, Kubernetes.

7. État d'avancement et prochaines étapes

Code : aucun code n'a été écrit à ce stade. Il n'y a que des formalisations mathématiques (sections 4 et 6).

Acquis : problème choisi (n°8), expliqué avec exemple ; état de l'art par familles avec limites ; 5 perspectives ; solution en une phrase (proposée) ; thème de recherche rédigé (proposé).

Non encore tranché : je n'ai pas encore confirmé le titre exact du thème, ni choisi définitivement entre les perspectives P1 à P5 (ou une combinaison). Le thème proposé est positionné entre algorithmique et systèmes ; il peut être recentré vers le théorique (P3) ou l'apprentissage (P4).

Prochaines étapes envisagées :

1. Valider ou ajuster le titre et la perspective retenue.
2. Établir un **plan de lecture** des articles prioritaires (Erms, Microscaler, Autothrottle, GRAF, FIRM, Sinan, Seer, Google Autopilot), avec vérification de chaque référence.
3. Pour chaque article : extraire l'algorithme, les avantages, les limites et les concepts utiles.
4. Affiner la question de recherche et les hypothèses.
5. Choisir le benchmark et préparer l'environnement expérimental (Kubernetes, TrainTicket ou DeathStarBench, générateur de charge).